

Smart Grid Digital Twin — Project Documentation

Adaptive Smart Grid Management via GenTwin & LLM Integration

Kalyanpur + Dhanmondi case study · Documentation as of 2026-07-05

1. What the project is

An explainable Smart Grid digital-twin pipeline for Dhaka. Real BPDB substation data drives a deep-learning day-ahead forecaster; the forecast triggers a multi-objective (MOLP) demand-response optimisation over a synthetic but BPDB-validated household population; high-impact events are filtered by a Priority Pool (GCS score) and loaded into a Neo4j knowledge graph, which grounds an LLM that explains every grid decision to the customer.

Real BPDB data (18 substations)

|



PHASE 1 — Research / model selection (offline, run once)

load_prediction.py : 6 DL architectures × 18 substations

split: train 2019–2023 | validation 2024 | test 2025–26

winner selected by mean VALIDATION R^2 (test used once, for reporting)

| winner → forecast_outputs/global_model_selection.csv



PHASE 2 — Operational pipeline (run_pipeline.py)

00 calendar → 01 weather → 03 composition → 04 load generator

→ 05 validation → 05b forecast (winner model, 2 substations)

→ 06 MOLP → 07 Priority Pool → 08 Knowledge Graph → Neo4j → LLM/XAI

Two-level design (do not mix): the 18 substations exist only to select the best architecture experimentally; Kalyanpur and Dhanmondi are the deployment case study. The winner is never hardcoded — 05b reads it from `global_model_selection.csv`.

Experimental result (leak-free protocol, full 108-model run): winner = **BiGRU** — mean Val R^2 0.6882, mean rank 1.83, 9/18 wins; reported-once mean Test R^2 0.7594. The old protocol's winner (BiGRU+BiLSTM, test-selected, 0.7456) was overturned — a reportable finding: the leaky protocol had picked the wrong architecture. Deployed case-study performance: Kalyanpur val 0.828 / test 0.853; Dhanmondi val 0.589 / test 0.786.

2. Script-by-script explanation

`config.py` — shared configuration

Single place for: paths (`data/`, `kg/`, BPDB CSV auto-discovery), time base (2024-01-01 → 2026-06-28, 30-min slots, seed 42), the two case-study substations, BPDB yearly peak targets, customer composition shares, tier mix and incomes, coincident peaks per building type, the 12 named Dhanmondi hospitals, substation metadata (utility, lat/lon), Bangladesh calendar (Ramadan/Eid/Durga Puja/public holidays 2024–26), and all MOLP/tariff parameters (TOU multipliers, tier tariff rates, SGO supply costs, tier α weights, discomfort costs, 0.5 max-curtail fraction, 90th-percentile stress quantile, 3 BDT/kWh DR rebate).

`00_bd_calendar.py` — Bangladesh calendar

Builds the 30-min calendar (43,728 rows). Day types collapsed to {Weekday, Weekend, Public Holiday, Eid, Durga Puja} mirroring the BPDB Holiday_type encoding; Fri/Sat weekend; Ramadan flags; a simple whole-day `load_multiplier` (Eid 0.78 — city empties — to Weekday 1.00).

`01_dhaka_weather.py` — synthetic 30-min weather

Daily anchors (Tmax/Tmin/Tmean/humidity) are taken from the **real** BPDB weather columns, then a half-cosine diurnal curve (min ~05:30, max ~14:30) fills each day; humidity moves inversely to temperature. Output is deliberately minimal: temperature, humidity, season.

`02_appliance_catalogue.json` — appliance catalogue (data, not a script)

21 verified Bangladesh residential appliances (Walton/Singer/Vision/Gree... power ratings). Per appliance: rated power $\pm$ std, duty cycle, DR category (Critical / Schedulable / Curtailable), usage class (comfort, cooking, always_on, ...), whether temperature-driven, and per-tier ownership probability

- quantity distribution. This is the ground truth 04 samples from and the source of the KG's appliance categories.

03_feeder_composition.py — who is on each feeder

Chooses the number of households (per tier), shops, offices, schools, clinics, named Dhanmondi hospitals, workshops, WASA pumps, street-light blocks and mosques so that the sum of design coincident peaks equals each substation's observed BPDB mean daily peak. Matching is by composition, never by profile scaling.

04_load_generator.py — bottom-up synthetic load

Simulates every detailed household appliance-by-appliance at 30-min resolution for 2.5 years (temperature-driven fans/AC, usage-class time profiles, calendar multipliers), aggregates all customer classes to the substation level, and calibrates per-year unit counts against BPDB (03b*_composition_calibrated.csv). Also exports the per-household category-split loads (.npz) that MOLP optimises.

05_validate_against_bpdb.py — validation

Daily synthetic peak vs real BPDB peak for every day: MAPE, correlation, yearly means, overlay + scatter plots. No post-hoc scaling — residuals reported as-is.

load_prediction.py — PHASE 1: model-selection experiment (leak-free)

Cleans the BPDB series (clock-time typo artefacts, >450 MW filter) and trains **six** architectures (BiLSTM, BiGRU, BiLSTM+BiGRU, BiGRU+BiLSTM, CNN+BiLSTM+BiGRU, CNN+BiGRU+BiLSTM) on **all 18 substations** under a **strict chronological split**: train 2019–2023, validation 2024, test 2025–26 (window 14, seed 42). Leak-free by construction: winsorisation quantiles and gap-fill Month×DOW medians are computed from **train years only**; interpolation is **forward-only** (causal); the MinMaxScaler is fit on train only; sequence context windows use only past data; early stopping and LR scheduling monitor the explicit 2024 validation set (`validation_data=`, never trained on). **Model selection is validation-based**: architectures are ranked by mean Validation R^2 (tie: mean val rank); test R^2 /RMSE/MAE are reported for every model but never influence the choice — the test set is used exactly once. Outputs: `global_model_selection.csv` + per-substation `metrics_summary.csv` (now with `val_r2/val_rmse/val_mae` columns). Run once, offline; substations whose record starts after 2023 fall back to a chronological 70/15/15 split.

05b_substation_forecast.py — PHASE 2: operational forecaster (NEW)

Reads the **winner** from `global_model_selection.csv` (hard-fails with instructions if Phase 1 hasn't run; `FORCE_MODEL` env override for experiments). Trains **ONLY** that architecture, **ONLY** for Kalyanpur and Dhanmondi, by **importing** `load_prediction`'s own

functions (cleaning, features, `prepare_train_test`, `train_one_model`) — the network is defined in exactly one place. Produces one-step-ahead daily peak forecasts over the whole record, saves the model (`.keras`) + scaler (`.pkl`) + forecast CSVs, and writes a **fingerprint** (BPDB file + selection file + winner + all hyper-parameters incl. the validation year) so forecasts regenerate automatically when data, the Phase-1 winner, or the protocol changes — and are skipped otherwise. Deployment protocol = Phase-1 protocol: trains on 2019–2023 only; 2024 is seen solely through early stopping (validation); 2025–26 fully out-of-sample — so **no day of the pipeline window 2024–2026 is a training day**.

06_molp_optimizer.py — MOLP demand response (REWRITTEN, TF-free)

Three linear objectives — customer cost, customer QoE (discomfort), SGO profit — scalarised with tier-specific α ; because objectives are linear with box constraints, the bang-bang threshold rule is the **exact** optimum. Decision variables per household $\times$ evening-peak slot: shift fraction of Schedulable load to off-peak, curtail fraction (≤ 0.5) of Curtailable load; Critical load is a hard must-run constraint. **SGO-level forecast integration:** grid-stress days are declared **day-ahead** from the forecasted substation peak (top 10% of 05b forecasts) — exactly "feed predicted demand into MOLP" from the proposal; fallback to the realised synthetic aggregate if the forecast CSV is missing. `predicted_load_w` in the event log is the household's simulated counterfactual baseline; the substation forecast is logged per event as `substation_pred_peak_mw`. Contains no deep-learning code. Also produces the Pareto α -sweep, peak-day plots, the SGO summary, **per-household monthly bills** (`06_<sub>molp_monthly_bills.csv`: baseline vs optimised bill net of DR rebate, saving, energy, discomfort — feeds the KG's MonthlyBill nodes) and the **daily stress calendar** (`06_<sub>stress_calendar.csv`: forecast vs threshold for EVERY day — feeds ForecastDay nodes so the LLM can also explain non-events, "why was there NO demand response").

07_priority_pool.py — Priority Pool / GCS gating

$GCS = 0.4 \cdot PDR + 0.3 \cdot PMN + 0.3 \cdot AA$ (threshold 0.50). Keeps only grid-stress, high-criticality events (context reduction for the LLM), serialises the top 500 as grounded JSON payloads for the Twin Adapter / LLM Reasoning Engine (now including `substation_pred_peak_mw` in `grid_context`), and reports context-reduction statistics + the GCS histogram.

08_knowledge_graph.py — Neo4j knowledge graph builder

Builds all node/relationship CSVs (bulk-import format), a `load_kg.cypher` loader and `llm_context.md` (GraphRAG usage guide). DrEvent nodes carry `substation_pred_peak_mw` and `temp_max_c` (that day's max temperature), and their explanation cites the forecast. The KG now also builds: **MonthlyBill nodes** (`(:User)-[:BILLED {month}]->`) with baseline/optimised bill, saving, discomfort AND the month's weather context (`mean_temp_c`, `max_temp_c`, `hot_days` $\geq 33^\circ\text{C}$, `mean_humidity_pct`, `rain_mm` from real BPDB precipitation); **ForecastDay nodes**

((:ForecastDay)-[:FORECAST_FOR]->(:Substation)) — every day's forecast, stress threshold, stress flag and max temp, grounding "why no DR that day"; and `avg_monthly_bill_bdt` on User nodes. Weather is loaded via `load_weather_context()` with graceful skip if the CSV is missing. All writes are explicit UTF-8 (Windows cp1252 crash fix).

Run_research.py — PHASE-1 runner (NEW)

Banner + `load_prediction.py` in one command; the experiment prints per-model lines (epochs, duration, Train/Val/Test R²) and per-substation elapsed+ETA; TensorFlow C++ log spam is suppressed. Full run ≈ 5.8 h on the user's CPU.

regenerate_selection.py — crash recovery (NEW)

Rebuilds every post-training Phase-1 artefact (selection CSV/figure, R² boxplot, per-substation report, `global_best_model.txt`) from the saved `metrics_summary.csv` in ~2 s — no retraining. Used after the matplotlib ≥3.9 `boxplot(labels=)` crash (now fixed version-adaptively in `load_prediction`).

Run_pipeline.py — orchestrator

Runs 00 → 01 → 03 → 04 → 05 → **05b** → 06 → 07 → 08 in order. Header documents that Phase 1 (`load_prediction.py`) is an offline prerequisite, not a pipeline step. A **pre-flight guard** (mirrored inside 05b) aborts immediately if `global_model_selection.csv` is missing or from the old protocol (no `mean_val_r2` column) — an obsolete winner can never be deployed silently.

3. Data files (data/ + inputs) — attributes

BPDB_Dhaka_City_Substations_Page3.csv (INPUT — real data)

One row per day (2019 → 2026-06-28). Key columns: `Date`; `<Substation>_Load_MW` (daily peak load, 18 substations — contains clock-time typo artefacts like 1830 and implausible >450 MW values, cleaned in code); `<Substation>_Peak_Time`; weather (`Temp_Max_C`, `Temp_Min_C`, `Temp_Mean_C`, `Humidity_Mean_Pct`, `Precip_Sum_mm`, `Rain_Sum_mm`, `Wind_Max_kmh`); `Holiday_type` {No Holiday, Public Holiday, Eid, Durga Puja}.

00_bd_calendar_30min.csv

Column	Meaning
timestamp	30-min timestamp
date, hour, minute	components
day_of_week , day_name	0=Mon...6=Sun
is_weekend	1 = Friday/Saturday (BD weekend)
is_ramadan	1 during Ramadan spans
day_type	Weekday / Weekend / Public Holiday / Eid / Durga Puja
is_holiday	1 for the three holiday classes
load_multip lier	whole-day demand factor (Eid 0.78 ... Weekday 1.00)

01_dhaka_weather_30min.csv

timestamp, temperature_c (diurnal curve anchored on real BPDB daily values + noise), relative_humidity_pct (inverse to temperature), season {winter, summer, monsoon, autumn}.

03_<sub>_composition.csv

substation, category (residential/commercial/educational/health/industrial/infrastructure), subcategory (tier name, shop, school, hospital::<name>, hospital::<name>),

was_a_pump...), units, per_unit_peak_kw, design_peak_kw, share_of_design_peak_pct.

03b_<sub>_composition_calibrated.csv (written by 04)

Same rows plus units_2024, units_2025, units_2026 — per-year unit counts calibrated so the synthetic aggregate matches each year's BPDB mean daily peak. Used by 06's substation_scale() to extrapolate the 120-HH sample to the full residential cohort.

04_<sub>_households_30min.csv

Wide matrix: timestamp + one column per detailed household (e.g. KHH0001) with total load in W.

04_<sub>_hh_category_loads.npz (NumPy archive)

household_ids, tiers, and three ($T \times n_{hh}$) arrays in W: critical_w, schedulable_w, curtailable_w — the exact MOLP inputs.

04_<sub>_household_inventory.csv

One row per appliance unit: household_id, substation, tier, household_size, monthly_income_bdt, meter_id, appliance_id, appliance_name, unit_index, category (Critical/Schedulable/ Curtailable), rated_power_w, actual_power_w, duty_cycle, quantity. Feeds both the KG (Appliance nodes) and 07's payload appliance lists.

04_<sub>_aggregate_30min.csv

timestamp, one <class>_kw column per customer class (residential tiers, shops, hospitals, pumps...), total_mw — the synthetic substation load.

04_<sub>_appliance_sample_7days.csv

Long-format appliance traces for one Ramadan-start week (11–17 Mar 2024): timestamp, household_id, appliance_name, category, power_w.

05_daily_peak_validation.csv

Per day and substation: synthetic_peak_mw, bpdb_peak_mw, error_mw, abs_pct_error — the bottom-up validation table.

05b_<sub>_daily_forecast.csv (NEW — Phase-2 output)

C o l u m n	Meaning
d a t e	calendar day
a c t u a l - m w	cleaned BPDB daily peak
p r e d - m w	one-step-ahead forecast by the selected model (BiGRU+BiLSTM)

05b_forecast_meta.json (NEW)

`fingerprint` (cache key), `selected_model`, `phase1_selection_file`, hyper-parameters, and per-substation deployment test metrics.

06_<sub>_molp_events_log.csv (household × day events)

Column	Meaning
timestamp	event day
household_id, substation, tier	who
tou_period, tou_multiplier, tou_rate_bdt_kwh	tariff context (peak, 1.35)
predicted_load_w	simulated counterfactual baseline evening-peak load
optimised_load_w	post-MOLP evening-peak load
power_deviation_w , power_deviation_r atio	MOLP throttle; $PDR = (base - opt) / base$, the GCS input
price_multiplier_norm	price-shock signal (1.0 stress day, 0.5 otherwise)
appliance_alpha	priority of highest affected category (Curtable 0.7 > Schedulable 0.4)
molp_alpha	tier fairness weight α used in the scalarisation

<code>action_taken</code>	SHIFT / CURTAIL+SHIFT / DR_CURTAIL+SHIFT
<code>is_stress_day</code>	1 if the forecast declared a DR day
<code>energy_shifted_kwh,</code> <code>energy_curtailed_kwh</code>	DR energy that day
<code>substation_pred_peak_mw</code>	day-ahead BiGRU+BiLSTM substation forecast (NEW; replaces the removed <code>ensemble_w_bilstm/ensemble_w_bigru</code>)

06_<sub>_molp_monthly_bills.csv (NEW)

Per household × month (3,600 rows/substation): `baseline_bill_bdt`, `optimised_bill_bdt` (net of rebate), `saving_bdt`, `saving_pct`, `energy_kwh`, `energy_shifted_kwh`, `energy_curtailed_kwh`, `discomfort_bdt_eq`.
Conservation-tested (monthly sums == slot totals).

06_<sub>_stress_calendar.csv (NEW)

Per day: `pred_peak_mw`, `stress_threshold_mw` (90th pct of forecasts), `is_stress_day` — the SGO's complete day-ahead DR decision record.

06_<sub>_molp_household_summary.csv

Per household: `alpha`, `baseline_cost_bdt`, `optimised_cost_bdt`, `saving_pct`, `energy_shifted_kwh`, `energy_curtailed_kwh`, `discomfort_bdt_eq`, `qoe_score`.

06_molp_pareto.csv / 06_sgo_summary.csv

α sweep (saving % vs discomfort vs SGO profit) tracing the Pareto frontier; per substation × year: `evening_peak_reduction_mw`, `sgo_profit_delta_bdt`, `n_stress_days`.

07_optimization_events_log.csv

The full 06 event log (both substations) + `gcs_score`.

07_priority_pool_high_impact.csv

Only stress-day events with GCS > 0.50, sorted by GCS — the events that reach the LLM/KG layer. Same columns as above.

07_llm_payload.jsonl

Top-500 events as grounded JSON blocks: `grid_context` (substation, date, TOU, stress flag, price shock, **substation_pred_peak_mw**), `molp_decision` (predicted/optimised W, throttle, action, α , kWh, rebate), `household_context` (tier, tariff, appliance lists per category), and the LLM task instruction.

07_priority_pool_stats.csv

`molp_events_total`, `high_impact_events`, `context_reduction_pct`, `gcs_threshold`, mean GCS all/pool, % of pool on stress days.

Phase-1 outputs (**forecast_outputs/**)

`metrics_summary.csv` — one row per model × substation (+ next-day columns): train/val/test R^2 , RMSE, MAE. `global_model_selection.csv` — one row per model: `model`, `mean_val_r2`, `min_val_r2`, `mean_rank` (validation-based), `wins` (validation-based), `mean_val_rmse`, then reporting columns `mean_test_r2`, `min_test_r2`, `mean_test_rmse`, `mean_test_mae`, `mean_train_r2`; **row 1 = winner by mean Validation R^2** , which 05b reads. Regenerate by re-running `python load_prediction.py` — old-protocol tables must not be cited.

4. Knowledge-graph files (**kg/**) — attributes

Schema:

(:User)-[:HAS_METER]->(:Meter)-[:FEEDS_FROM]->(:Substation)

(:Meter)-[:LOCATED_IN]->(:Ward)-[:PART_OF]->(:Substation)

(:User)-[:IN_TIER {income_bdt}]->(:SocioeconomicTier)

(:Meter)-[:REGISTERS]->(:Appliance)-[:INSTALLED_AT]->(:Meter)

(:Appliance)-[:BELONGS_TO {flexibility_index}]->(:Category)

(:User)-[:SUBJECT_TO {monthly_kwh_est}]->(:TariffSlab)

(:User)-[:EXPERIENCED {date}]->(:DrEvent)-[:DURING]->(:TouPeriod)

(:User)-[:BILLED {month}]->(:MonthlyBill)

(:ForecastDay)-[:FORECAST_FOR]->(:Substation)

Node files (each has **nodeId** — the unique key — and **label**)

nodes_User.csv (240): household_id, tier, household_size, monthly_income_bdt, substation.

nodes_Meter.csv (240): meter_id, household_id, meter_type (smart_prepaid), substation_id, ward_id, monthly_kwh_est.

nodes_Appliance.csv (2,973): appliance_id, appliance_name, category, rated_power_w, actual_power_w, duty_cycle, quantity, household_id.

nodes_Substation.csv (2): name, city, division, utility (DESCO/DPDC), mean_daily_peak_mw_2024/25/26 (real BPDB yearly means), feeder_voltage_kv, latitude, longitude.

nodes_Ward.csv (10): name, substation_id — 5 supply wards per substation.

nodes_Category.csv (3): name, curtailment_priority (1 first...3 never), can_curtail, can_schedule, molp_constraint (must_run / flexible_start / power_reduction_allowed), explanation_template — the customer-facing NL template with an {appliance} placeholder that grounds the LLM.

nodes_TouPeriod.csv (3): name, hours (peak 17:00–23:00 etc.), multiplier (1.35 / 1.00 / 0.80).

nodes_SocioeconomicTier.csv (3): name, income_min_bdt, income_max_bdt.

nodes_TariffSlab.csv (7): BERC Feb-2024 slabs — slab_id, from_kwh, to_kwh, rate_bdt_kwh (4.63–14.61), tou_peak_rate, tou_offpeak_rate.

nodes_DrEvent.csv (800): date, substation, action, energy_shifted_kwh, energy_curtailed_kwh, molp_alpha, gcs_score, predicted_load_w (household counterfactual baseline), optimised_load_w, power_deviation_ratio, **substation_pred_peak_mw** (NEW — the day-ahead forecast that triggered the DR window), temp_max_c (that day's max temperature), explanation (rendered sentence citing the forecast — the RAG fallback).

nodes_MonthlyBill.csv (7,200 = 240 HH × 30 months): household_id, month, baseline_bill_bdt, optimised_bill_bdt, saving_bdt, saving_pct, energy_kwh, energy_shifted_kwh, energy_curtailed_kwh, discomfort_bdt_eq,

plus month weather: `mean_temp_c`, `max_temp_c`, `hot_days`, `mean_humidity_pct`, `rain_mm`.

nodes_ForecastDay.csv (1,820 = 910 days × 2 substations): `date`, `substation`, `pred_peak_mw`, `stress_threshold_mw`, `is_stress_day`, `temp_max_c` — grounds explanations of DR **absence** as well as presence.

User nodes additionally carry `avg_monthly_bill_bdt`.

Relationship files (all have `:START_ID`, `:END_ID`, `:TYPE`)

File	Connections	Extra property
<code>rels_HAS_METER.csv</code>	User → Meter	—
<code>rels_FEEDS_FROM.csv</code>	Meter → Substation	—
<code>rels_LOCATED_IN.csv</code>	Meter → Ward	—
<code>rels_PART_OF.csv</code>	Ward → Substation	—
<code>rels_IN_TIER.csv</code>	User → Tier	<code>income_bdt</code>

rels_SUBJECT_T0.csv	User → TariffSlab	monthly_kwh_est
rels_REGISTERS.csv	Meter → Appliance	—
rels_INSTALLED_AT.csv	Appliance → Meter	—
rels_BELONGS_T0.csv	Appliance → Category	flexibility_index (~0.05 Critical, ~0.45 Curtailable, ~0.70 Schedulable)
rels_EXPERIENCED.csv	User → DrEvent	date (⚠ DD-MM-YY vs node's YYYY-MM-DD — normalised in the fixed loader)
rels_DURING.csv	DrEvent → TouPeriod	—
rels_BILLED.csv	User → Monthly Bill	month

<code>rels_FOREC</code>	Forecast	—
<code>AST_FOR.cs</code>	Day →	
<code>v</code>	Substation	

Loader: use `load_kg_full.cypher` (MERGE-based, re-runnable, per-label constraints, no `:Node` label, all properties typed, dates normalised) — add `substation_pred_peak_mw` to its DrEvent block after regenerating the KG CSVs.

5. Changelog of this working session

1. **BiGRU+BiLSTM integration** — the global-best model replaced the statistical proxy predictor (anomalies found & reported first: no saved checkpoints; substation-vs-household granularity mismatch; meaningless ensemble-weight columns; 0.0015-R² selection margin over BiGRU; 2024 in-sample leakage; new TF dependency).
2. **SGO-level refactor** (supervisor/GPT feedback) — forecasting moved out of 06 into the new `05b_substation_forecast.py`; the MW→household-W bridge and its arbitrary clip deleted; grid-stress days now declared day-ahead from the substation forecast; `substation_pred_peak_mw` added through 06 → 07 → 08 into the KG; 06 is TensorFlow-free.
3. **Two-phase methodology** — 05b now **reads** the winner from `global_model_selection.csv` (hard-fails if Phase 1 hasn't run), names artefacts after the winner, and its cache fingerprint invalidates on data change, config change, or a new Phase-1 winner. `load_prediction.py` remains untouched as the 6-model × 18-substation experiment.
4. **Leak-free experimental methodology** — strict chronological split (train 2019–2023 / validation 2024 / test 2025–26); winsorisation and imputation statistics from train years only; causal (forward-only) interpolation; explicit validation set for early stopping; **model selection moved from test to validation metrics** (test used exactly once, for reporting). Only `load_prediction.py` and `05b_substation_forecast.py` changed; the old 0.7456 table is obsolete and Phase 1 must be re-run.
5. **Monthly bills in the KG** — 06 computes per-household monthly bills (conservation-verified); 08 loads 7,200 MonthlyBill nodes + BILLED relationships + User.avg_monthly_bill_bdt, making "how much did I save last month, at what comfort cost?" a single-hop lookup.
6. **Weather-grounded explainability** — MonthlyBill nodes carry the month's `mean/max temp`, `hot_days`, `humidity`, `rain_mm` (real BPDB precipitation); DrEvent and ForecastDay nodes carry that day's `temp_max_c` — the LLM can now

say "June had 22 hot days vs 9 in May, so your AC ran more and the bill rose" with every number from the graph.

7. **ForecastDay nodes + stress calendar** — every day's forecast vs the stress threshold is in the graph, so the system explains DR **absence** ("no DR yesterday: forecast 88 MW < threshold 114.6 MW"), not just presence. 11-query GraphRAG retrieval pack written ([llm_retrieval_queries.md](#)) incl. peer comparison and GCS-priority decomposition; honest out-of-scope limits: exact per-appliance kWh (NILM) and finer weather effects.
8. **Phase-1 executed & recovered** — full 108-model run completed (5.8 h): **BiGRU selected** (Val R^2 0.6882, 9/18 wins), overturning the old protocol's BiGRU+BiLSTM; matplotlib ≥ 3.9 boxplot crash fixed version-adaptively; [regenerate_selection.py](#) recovers post-training artefacts without retraining; old-protocol guards added to Run_pipeline and 05b; TF log spam silenced; per-model/per-substation progress + ETA added to the trainer.
9. **Neo4j tooling** — step-by-step Neo4j guide, relationship-inspection queries, and the clean re-runnable [load_kg_full.cypher](#) (fixes the `:Node`-label hairball and the constraint-violation error).

Verified in-session: 05b end-to-end (train → save → cache → skip → invalidate on winner/data change), 06 forecast-driven stress days (exactly 10%, Apr–Sep — Dhaka's real peak season), 07/08 edits, all syntax. Full 150-epoch metrics will be recorded in [05b_forecast_meta.json](#) on your first real pipeline run.